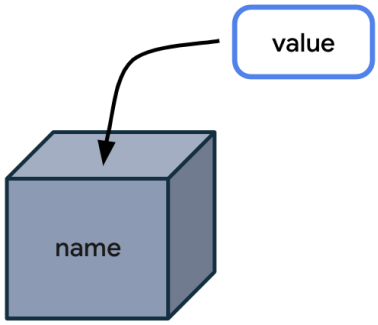
VARIABLES, DATA TYPES AND OPERATORS

Variables – Introduction

In Java, Variables are the data containers that stores the data values during Java program execution. Every Variable in Java is assigned a data type that designates the type and quantity of value it can hold. A variable is a memory location name for the data.

The value stored in a variable can be changed during program execution. Variables in Java are only a name given to a memory location. All the operations done on the variable affect that memory location. In Java, all variables must be declared before use.



Rules for declaring a Variable

- A variable name can consist of Capital letters A-Z, lowercase letters a-z digits 0-9, and two special characters such as _ underscore and $ dollar sign.
- The first character must not be a digit.
- Blank spaces cannot be used in variable names.
- Java keywords cannot be used as variable names.
- Variable names are case-sensitive.
- There is no limit on the length of a variable name but by convention, it should be between 4 to 15 chars.
- Variable names always should exist on the left-hand side of assignment operators.

How to declare a Variable

In Java we can declare variables in three ways, discussed in https://replit.com/@ManuChaitanya/variablesDataTypesOperations#Main.java

Data Types – Introduction

Java provides two types of data representation: primitive types and reference types. Java has eight built-in data types, referred to as Java primitive types; variables of this type hold their values directly. Reference types hold references to objects (instances of classes). Unlike primitive types that hold their values in the memory where the variable is allocated, references don't hold the value of the object they refer to.

Instead, a reference points to an object by storing the memory address where the object is located.

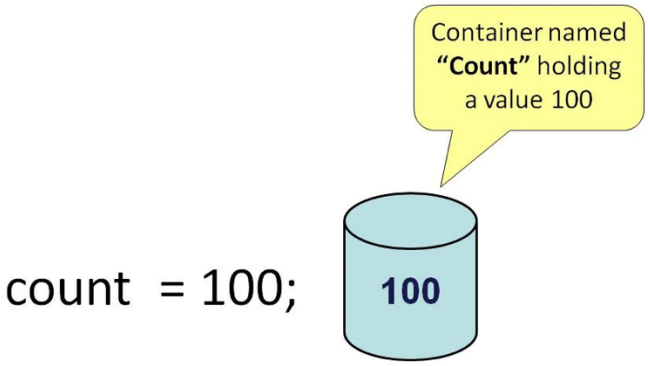
The eight primitives defined in Java are: int, byte, short, long, float, double, Boolean and char. These aren't considered objects and represent raw values.

They're stored directly on the stack. First we declare the data type of literal, there only the n byte space gets vacant in stack memory. Now the literal occupies the space and gets hold by a variable.

We'll take a look at storage size, default values and examples of how to use each type:

Type	Size (bits)	Minimum	Maximum	Example
byte	8	-2^{7}	$2^{7}-1$	*byte b = 100;*
short	16	-2^{15}	$2^{15}-1$	*short s = 30_000;*
int	32	-2^{31}	$2^{31}-1$	*int i = 100_000_000;*
long	64	-2^{63}	$2^{63}-1$	*long l = 100_000_000_000_000;*
float	32	-2^{-149}	$(2-2^{-23})\cdot 2^{127}$	*float f = 1.456f;*
double	64	-2^{-1074}	$(2-2^{-52})\cdot 2^{1023}$	*double f = 1.456789012345678;*
char	16	0	$2^{16}-1$	*char c = 'c';*
boolean	1	–	–	*boolean b = true;*

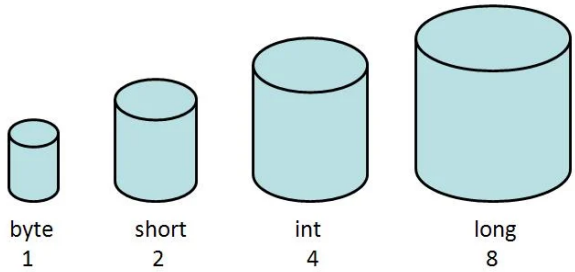
- **Int:** The first primitive data type we're going to cover is int. Also known as an integer, int type holds a wide range of non-fractional number values. Specifically, Java stores it using 32 bits of memory. In other words, it can represent values from -2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31}-1$). In Java 8, it's possible to store an unsigned integer value up to 4,294,967,295 ($2^{32}-1$) by using new special helper functions.



- **Byte:** It is a primitive data type like int, except it only takes up 8 bits of memory. This is why we call it a byte. Because the memory size is so small, byte can only hold the values from -128 (-2^7) to 127 ($2^7 - 1$).

- **Short:** The next stop on our list of primitive data types in Java is short. If we want to save memory and byte is too small, we can use the type halfway between byte and int: short. At 16 bits of memory, it's half the size of int and twice the size of byte. Its range of possible values is -32,768(-2^{15}) to 32,767($2^{15} - 1$).

- **Long:** Our last primitive data type related to integers is long. long is the big brother of int. It's stored in 64 bits of memory, so it can hold a significantly larger set of possible values. The possible values of a long are between -9,223,372,036,854,775,808 (-2^{63}) to 9,223,372,036,854,775,807 ($2^{63} - 1$).



- **Float**: We represent basic fractional numbers in Java using the float type. This is a single-precision decimal number. This means that if we get past six decimal points, the number becomes less precise and more of an estimate. In most cases, we don't care about the precision loss. But if our calculation requires absolute precision (e.g., financial operations, landing on the moon, etc.), we need to use specific types designed for this work. This type is stored in 32 bits of memory just like int. However, because of the floating decimal point, its range is much different. It can represent both positive and negative numbers. The smallest decimal is $1.40239846 \times 10^{-45}$, and the largest value is $3.40282347 \times 10^{38}$. And the default value is 0.0 instead of 0. Also, notice we add the f designation to the end of the literal number to define a float. Otherwise, Java will throw an error because the default type of a decimal value is double. We can also perform all standard arithmetic operations on floats. However, it's important to note that we perform floating point arithmetic very differently than integer arithmetic.

- **Double:** Next, we look at double. Its name comes from the fact that it's a double-precision decimal number. It's stored in 64 bits of memory. This means it represents a much larger range of possible numbers than float. Although, it does suffer from the same precision limitation as float does. The range is $4.9406564584124654 \times 10^{-324}$ to $1.7976931348623157 \times 10^{308}$. That range can also be positive or negative. The default value is also 0.0 as it is with float. Like float, we attach the letter D to designate the literal as a double.

- **Boolean**: The simplest primitive data type is Boolean. It can contain only two values: true or false. It stores its value in a single bit. However, for convenience, Java pads the value and stores it in a single byte. Declaring it without a value default to false. Boolean is the cornerstone of controlling our programs flow. We can use Boolean operators on them (e.g., and, or etc.).

- Char: The final primitive data type to look at is char. Also called a character, char is a 16-bit integer representing a Unicode-encoded character. Its range is from 0 to 65,535. In Unicode, this represents '\u0000' to '\uffff'.

  For a list of all possible Unicode values, check out sites such as Unicode Table.

Overflow and Underflow

Overflow- The primitive data types have size limits. But what happens if we try to store a value that's larger than the maximum value?

We run into a situation called overflow. When an integer overflows, it rolls over to the minimum value and begins counting up from there.

Floating point numbers overflow by returning Infinity.

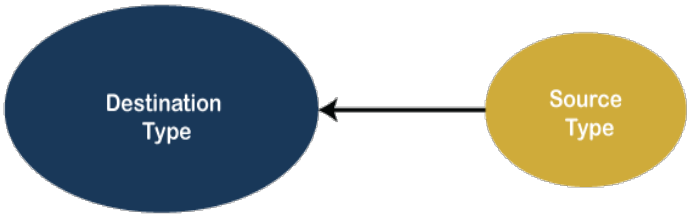
Underflow is the same issue except it involves storing a value smaller than the minimum value. When the numbers underflow, they return 0.0.

Compatibility

- Byte is compatible with short, int, long, float, and double.
- Short is compatible with int, long, float, and double.
- Char is compatible with int.
- Int is compatible with long, float, and double.
- Long is compatible with float and double.
- Float is compatible with double.

Data Type Conversion

Data Type is converted either automatically or manually.



- Automatic Type Conversion: Simply known as type conversion or implicit type conversion done by the compiler itself. There are two condition so that the complier can convert the data type:
  1. Both data types should be compatible with each other.
  2. The destination data type should not be smaller than the source type.

Suppose, we have an **int** data type and want to convert it into a **float** data type. These are data types compatible with each other because their types are numeric, and the size of int is 2 bytes which is smaller than float data type. Hence, the compiler automatically converts the data types without losing or truncating the values.

Conversions possible via Automatic Type Conversion:

- **Byte** to short, int, long, float, or double.
- **Short** to int, long, float, or double.
- **Char** to int, long, float, or double.
- **Int** to long, float, or double.
- **Long** to float or double.
- **Float** to double.

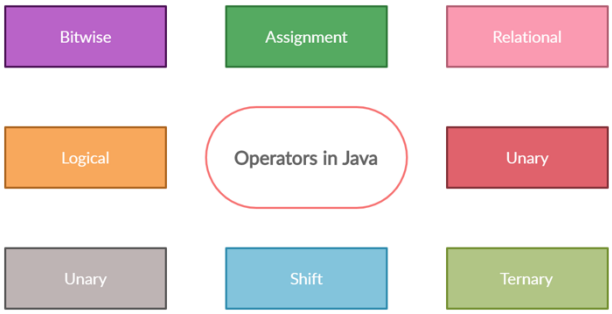
- **Manual Type Conversion:** Simply known as Type Casting, done by the programmer using casting() operator. It can be used both compatible data type and incompatible data type. When casting one data type to another, the destination data type must be smaller than the source data.

Operators – Introduction

Operators are a fundamental building block of any programming language. We use operators to perform operations on values and variables.

Java provides many groups of operators. They are categorized by their functionalities.

- Unary Operator
- Arithmetic Operator
- Shift Operator
- Relational Operator
- Bitwise Operator
- Logical Operator
- Ternary Operator
- Assignment Operator



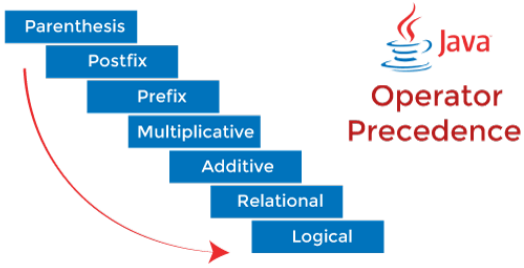
Operator Type	Category	Precedence
Unary	postfix	expr++, expr--
	prefix	++expr, --expr, +expr, -expr, ~, !
Arithmetic	multiplicative	*, / ,%
	additive	+, -
Shift	shift	<< ,>>, >>>
Relational	comparison	<, >, <=, >=, instanceof
	equality	==, !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	=, +=, -=, *=, /= ,%= ,&=, ^= ,|= ,<<= ,>>= ,>>>=

Precedence and Associativity of Operators

The **operator precedence** represents how two expressions are bind together. In an expression, it determines the grouping of operators with operands and decides how an expression will evaluate. While solving an expression two things must be kept in mind the first is a **precedence** and the second is **associativity**.

Precedence is the priority for grouping **different types of operators with their operands**. It is meaningful only if an expression has more than one operator with higher or lower precedence. The operators having higher precedence are evaluated first. If we want to evaluate lower precedence operators first, we must group operands by using parentheses and then evaluate.

We must follow **associativity** if an expression has **more than two operators of the same precedence**. In such a case, an expression can be solved either **left-to-right** or **right-to-left,** accordingly.



Precedence	Operator	Type	Associativity
15	() [] .	Parentheses Array subscript Member selection	Left to Right
14	++ --	Unary post-increment Unary post-decrement	Right to left
13	++ -- + - ! ~ (type)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
12	* / %	Multiplication Division Modulus	Left to right
11	+ -	Addition Subtraction	Left to right
10	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right
9	< <= > >= instanceof	Relational less than Relational less than or equal Relational greater than Relational greater than or equal Type comparison (objects only)	Left to right
8	== !=	Relational is equal to Relational is not equal to	Left to right
7	&	Bitwise AND	Left to right
6	^	Bitwise exclusive OR	Left to right
5	|	Bitwise inclusive OR	Left to right
4	&&	Logical AND	Left to right
3	||	Logical OR	Left to right
2	? :	Ternary conditional	Right to left
1	= += -= *= /= %=	Assignment Addition assignment Subtraction assignment Multiplication assignment Division assignment Modulus assignment	Right to left